

Resume as Code: Markdown Source, Typeset PDF, Multi-Variant Builds

Your resume is a document you'll edit dozens of times across years, ship to dozens of recipients, and need in three different formats (PDF, docx, plain text for ATS). Treat it like code: plain-text source, automated rendering, version control, branch-per-application. Word documents and Canva designs are dead ends. This how-to is the live one.

NOTE. LAST VALIDATED: 2026-05. pandoc 3.1+, TeX Live 2025 (or MiKTeX equivalent), WeasyPrint 62+, Python 3.12+ with [uv](#), AltaCV v1.7+, moderncv 2.0+. Bump and re-validate annually per [CLAUDE.md](#) standard #5.

What you'll have at the end

- A `resume/` repo (or subdirectory) with your resume in plain markdown, version-controlled, diffable like any other code.
- A build script that renders **PDF (typeset), docx (for HR portals), and plain-text/HTML (for ATS)** from one source.
- A **base resume + per-job variants** pattern: branch or overlay file per application, no copy-paste drift.
- A Dev Container with pandoc + TeX + WeasyPrint baked in — reproducible builds on any machine without polluting your host with 4 GB of LaTeX packages.
- Git footer injection so every PDF carries its commit SHA and build date.
- A clean separation between the "human-readable, pretty" variant and the "ATS-survivable, plain" variant.

Prerequisites

- Comfort with markdown and the command line.
- Git installed locally — ideally per [git-for-solo-devs](#).
- Docker available for the Dev Container approach (per [dockerized-deployments](#)); alternatively, pandoc + TeX Live installed directly on your host.
- A GitHub account (or any git remote) for the **private** repo where your resume will live.

Table of contents

1. [Why a plain-text resume](#)
2. [Resume vs CV: scope of this doc](#)
3. [Three viable approaches](#)
4. [The pandoc + LaTeX approach \(deep dive\)](#)
5. [Project structure](#)

6. [The markdown source](#)
 7. [The build script](#)
 8. [Multiple variants for different applications](#)
 9. [ATS considerations](#)
 10. [docx output for HR portals](#)
 11. [Dev Container for resume building](#)
 12. [Git workflow for a resume](#)
 13. [Privacy: don't ship PII to a public repo](#)
 14. [Versioning and dating the output](#)
 15. [Anti-patterns](#)
 16. [Alternatives considered](#)
 17. [Cheat sheet](#)
-

1. Why a plain-text resume

If you've kept a resume in Word or Google Docs for more than a year, you have already lived all of the following:

- **You can't diff it.** A `.docx` is a zip of XML. A `.gdoc` is a pointer to Google's servers. Both are binary as far as git is concerned. You cannot answer "what did I change in this resume since last year?" in any reasonable way. You cannot review your own edits.
- **Versions drift.** "Resume-final.docx", "Resume-final-v2.docx", "Resume-anthropic.docx", "Resume-anthropic-FINAL.docx". By the third variant, the **summary paragraph on one has subtly drifted** from the others, you fixed a typo in the Skills section of one but not the others, and your contact info is out of date in two of them.
- **Formatting fights you.** Word's auto-formatting decides to convert your hyphens to em-dashes mid-sentence. A copy-paste from LinkedIn brings in invisible styles. The "Bullets" button does something different depending on which paragraph you're in. You spend more time fighting the renderer than writing content.
- **ATS systems mangle layout.** Applicant Tracking Systems parse uploaded resumes into structured fields. A pretty two-column Canva resume often parses as a single unreadable string. The recruiter never sees your work because the ATS gave you a 12% match score on a job you're qualified for.
- **Lock-in is real.** Canva, Resume.io, Zety, etc. hold your data hostage behind a subscription. Export to PDF is fine until they change the export format, paywall a template, or shut down. You don't own the source.

The plain-text answer:

- **Source is markdown** (or YAML). Git-diffable. Editable in any text editor. Survives forever.
- **Rendering is automated.** Pandoc + a LaTeX template produces typeset PDFs that look professional. The same source produces docx for HR portals and plain text for ATS scans.
- **Variants are branches or overlays**, not copies. The "Anthropic version" is the base resume plus an overlay file (or a branch with three commits). When you fix a typo in the base, every variant gets it for free.
- **History is git.** What did your resume look like when you applied to Google in 2024? `git log` will tell you. `git show` will show you. `git checkout` will rebuild it.

This isn't about LaTeX purism. It's about **owning your source** and **eliminating manual sync work** between variants. The how-to picks pandoc + LaTeX because it produces the best-looking PDFs without making you write LaTeX directly — but the principle is the principle regardless of which renderer you pick.

IMPORTANT. THE RESUME IS A LONG-LIVED DOCUMENT. You'll edit it across decades. Every minute spent on the infrastructure of editing it is paid back hundreds of times. Don't optimize for "I need a resume tomorrow"; optimize for "I will have a resume forever."

2. Resume vs CV: scope of this doc

A quick disambiguation, because the words get used interchangeably and they shouldn't.

Term	Audience	Length	Style	Typical reader
Resume (US industry)	Hiring managers, recruiters, ATS	1 page (2 max for senior)	Tight, targeted, achievements-led	A recruiter skimming for 20 seconds
CV (academia / international)	Hiring committees, grant panels	Multi-page (5-20+)	Comprehensive, chronological, publication-heavy	A committee evaluating fit over an hour
CV (UK / EU general)	Confusingly: often used to mean "resume"	1-2 pages	Like a US resume	Same as resume

This doc focuses on the US-industry resume. For an MSDS student or experienced engineer applying to industry roles, that's what you ship — one page, achievement-focused, tailored per role. The same toolchain produces an academic CV too; you just remove the "1 page" constraint and add `\publications` / `\presentations` / `\grants` sections. AltaCV and moderncv both handle academic CVs natively.

If you'll apply to both industry roles and academic positions over your career, **keep the base source resume-shaped** (achievements, tight) and treat the CV as a long-form variant that includes the resume's content plus academic extras. Don't try to maintain a CV-shaped base and shrink it for industry — the inversion is much harder.

3. Three viable approaches

There are three reasonable toolchains for "plain-text source → rendered resume." All produce a PDF you can hand to a recruiter; they differ in typography quality, install footprint, and how much they fight you.

Approach	Source	Renderer	Typography	Install footprint	Best for
pandoc + LaTeX template (recommended)	Markdown + YAML	LaTeX via pandoc	Excellent (Knuth-grade kerning, microtypography)	Large (TeX Live $\approx$ 4 GB)	Anyone who wants professional-grade typesetting and isn't deterred by a Dev Container
Markdown + WeasyPrint	Markdown	HTML/CSS via WeasyPrint	Good (CSS controls everything)	Small (<code>pip install weasyprint</code>)	Hosts where TeX is impractical; people who already know CSS
JSON Resume schema	YAML or JSON (per jsonresume.org)	Themes (Node-based)	Varies by theme	Small (<code>npm</code> / <code>bun</code>)	ATS-first workflows; people who want many off-the-shelf themes

Use pandoc + LaTeX. It produces the best-looking output. Modern LaTeX templates (AltaCV especially) give you a contemporary, ATS-aware layout out of the box. You never write LaTeX directly — pandoc converts your markdown to LaTeX using the template, you just edit markdown.

When to reconsider WeasyPrint:

- You're already running the `scripts/build-pdfs.py` pipeline (this repo does it for the how-tos) and don't want a second toolchain.
- You're on a host where installing TeX is genuinely impractical (constrained environment, no Docker).
- You prefer CSS to LaTeX and want full control over the layout.

The WeasyPrint output is good. It's not as crisp as LaTeX for things like justified text, line breaking, and small caps. For a resume — which is read once for ~20 seconds — the difference is real but not decisive.

When to reconsider JSON Resume:

- You want **many themes** to A/B test (jsonresume.org has 100+ community themes).
- You want a **strict schema** that makes structural mistakes impossible (the JSON Schema validates your input).
- You're feeding the resume into other tooling (a portfolio site, an LLM-driven personalization system) that benefits from structured data.

The JSON Resume schema is excellent. The downside is the **theme ecosystem is uneven** — some themes are abandoned, some have bugs, some are stuck on old Node versions. Pick a theme and accept that you may need to fork it. The default theme is fine; "Elegant" and "Stack Overflow" are popular picks.

4. The pandoc + LaTeX approach (deep dive)

Why this combo

Pandoc is the Swiss-army document converter — input is markdown (or many other formats), output is PDF, HTML, docx, EPUB, Org, RST, etc. For a resume, pandoc's two key features are:

1. **YAML metadata headers.** You put `name`, `email`, `address`, etc. in the top of the markdown file. Pandoc passes those to the template as variables.
2. **Custom templates.** Pandoc has its own template language. A LaTeX template defines the document class, packages, layout, and where each variable goes. You ship the template once and never touch the LaTeX again.

LaTeX does the actual typesetting. The result is the same quality of typography that academic papers and books are set in — true small caps, proper kerning, microtypography (sub-pixel adjustments), proper hyphenation. A pandoc-rendered PDF from a good template is **visibly more polished** than any HTML-to-PDF approach.

The template choice

There are two modern resume/CV templates worth your time:

Template	Style	Best for	Notes
AltaCV	Modern two-column with accent color and icons	Engineering / data / industry resumes	Active maintenance, gorgeous out of the box, designed for screen and print
moderncv	Several built-in styles ("classic", "banking", "casual")	Academic CVs, conservative industries	Ships with TeX Live, no extra install

Use AltaCV for industry resumes. It's the best-looking modern template. The two-column layout (sidebar with skills/links, main column with experience) is what hiring managers expect to see for an engineering resume in 2026.

CAUTION. The AltaCV two-column layout is **BEAUTIFUL FOR HUMANS BUT ROUGH ON ATS PARSERS**. Multi-column PDFs frequently get extracted in column-major order, which produces gibberish ("Skills: PythonAcme Corp 2020-2024 led team of 6Senior Engineer..."). This is why you maintain **TWO VARIANTS**: a pretty AltaCV variant for human readers, and a single-column ATS variant for upload portals. See §9.

When to reconsider AltaCV:

- You're applying to extremely conservative industries (BigLaw, traditional finance) where minimalist black-and-white is the norm. Use moderncv "banking" style instead.
- You're submitting an **academic** CV. moderncv handles publications and grants better.
- Your name has glyphs outside Latin-1 that AltaCV's default font doesn't ship with. Switch the template's `\setmainfont` to something with broader coverage (e.g., Charter or Source Serif Pro via `fontspec`).

How pandoc fits with AltaCV

AltaCV ships as a `.cls` file (LaTeX class). Normally you'd write LaTeX directly against the class. The pandoc move is:

1. **Author content in markdown** with a YAML header.
2. **Write a pandoc LaTeX template** that wraps the markdown content with AltaCV's class and structural commands.
3. **Build:** `pandoc resume.md --template=templates/altacv.tex -o dist/resume.pdf --pdf-engine=xelatex`.

The template is roughly 80 lines of LaTeX you write **once** (or copy from a community example). After that, all editing is in markdown.

Installing the pieces

The Dev Container (\$11) is the recommended install path — it ships everything pre-baked. If you want it on the host:

```
# Pandoc – version 3.1+
sudo apt-get install pandoc

# TeX Live – the "full" install is ~5 GB but covers everything
sudo apt-get install texlive-full

# AltaCV – clone into your local TEXMF tree
mkdir -p ~/texmf/tex/latex/altacv
git clone https://github.com/liantze/AltaCV.git /tmp/AltaCV
cp /tmp/AltaCV/altacv.cls ~/texmf/tex/latex/altacv/
texhash ~/texmf
```

The `texlive-full` package is large; if disk is tight, `texlive-xetex texlive-fonts-extra texlive-latex-extra` is the minimum AltaCV needs.

5. Project structure

Use this layout. It's the same structure several of the more polished community resume projects converge on, and it cleanly separates source from output and base from variants.

```

resume/
├── resume.md                # canonical base source (markdown + YAML)
├── variants/
│   ├── sde-roles.md        # overlay for SDE/SWE-focused applications
│   ├── data-science.md    # overlay for DS/ML roles
│   ├── product-manager.md  # overlay for PM roles
│   └── ats-plain.md        # single-column overlay for ATS uploads
├── templates/
│   ├── altacv.tex          # pandoc LaTeX template wrapping AltaCV
│   └── ats.tex             # minimal single-column LaTeX template
├── scripts/
│   ├── build.py            # build script (pandoc invocation)
│   └── merge_overlay.py    # combines base + overlay into a build input
├── dist/                   # generated PDFs, docx, html – gitignored
├── .devcontainer/
│   ├── devcontainer.json
│   └── Dockerfile
├── .gitignore
├── Makefile                # `make pdf`, `make all`, `make sde`, etc.
└── README.md              # project notes (not the resume)

```

Why this layout:

- **resume.md is the single source of truth.** Every variant references it. If you fix a typo there, every variant gets it.
- **variants/ holds overlays, not duplicates.** An overlay is a markdown file that specifies which sections to replace or add. The merge step combines base + overlay; the result is what gets rendered. No copy-paste between files.
- **templates/ is rarely touched.** You write the LaTeX template once. After that, content edits stay in markdown.
- **dist/ is gitignored.** PDFs are build artifacts. Commit the source, not the output. (Exception: if your repo is private and you want to push the rendered PDF for hosting purposes, keep it — but most of the time `dist/` should be ignored.)
- **Makefile** gives you tab-completable, memorable commands: `make sde` produces the SDE variant.

`.gitignore`

```

dist/
*.aux
*.log
*.out
*.synctex.gz
.DS_Store
__pycache__/
*.pyc
.venv/
# Local-only personal data, if you use the public-repo + private-overlay pattern (§13)
personal.yaml

```

6. The markdown source

The markdown file has two parts: a **YAML front matter** block with metadata (your name, contact info, links) and a **body** with the resume sections written as markdown.

YAML front matter

```
---
name: "[Your Name Here]"
tagline: "Software Engineer / Data Scientist"
photo: false                # AltaCV supports an optional photo; usually off
email: "[you@example.com]"
phone: "[+1 555 555 0100]"
location: "[City, State]"
homepage: "[yourname.dev]"
github: "[github.com/yourhandle]"
linkedin: "[linkedin.com/in/yourhandle]"
orcid: ""                   # optional, for academic CVs
accent-color: "0E5484"      # AltaCV accent color, hex without "#"
# Optional: pandoc metadata
date: ""                    # leave blank; the build script injects it
git-sha: ""                 # leave blank; the build script injects it
---
```

The YAML keys map 1:1 to template variables. If you add a new key to the YAML, you reference it in the template as `$key$` (pandoc's template syntax).

The body

Markdown with `##` for sections. Use the structure below — it's the conventional shape for an industry resume and matches what AltaCV expects.

Summary

One short paragraph (2-3 sentences) framing who you are and what you bring. Written in the third person implicit ("Engineer with 8 years building...") not first person. ****Tailor this per variant.**** This is the single most job-specific section.

Example: "Senior software engineer with 8 years building distributed systems in Python and Go. Active-duty Army aviator transitioning to civilian data engineering. Master's in Data Science (UT Austin, in progress). Recent focus on streaming pipelines and ML infrastructure."

Experience

Senior Software Engineer – Acme Corp · 2022 – Present

Remote · Python · PostgreSQL · AWS

- Led the rebuild of the order pipeline from a single Django monolith into five focused services. Reduced p99 latency from 1,400ms to 220ms, cut monthly AWS spend 38% by killing redundant ECS tasks.
- Designed and shipped the company's first automated migration system (Alembic + GitHub Actions); 200+ migrations in production with zero downtime since rollout.
- Mentored two junior engineers from L3 to L4 over 18 months.

Software Engineer – Beta Inc · 2019 – 2022

Austin, TX · Go · Kubernetes · Postgres

- Built the realtime metrics ingestion service handling 1.2M events/sec at peak. Open-sourced the throttling library; 1,400+ GitHub stars.
- Reduced on-call pages by 70% by writing a runbook generator that pulled from our alerts catalog and produced per-service docs automatically.

Aviator – U.S. Army · 2007 – 2019

[Locations] · Operational and instructional roles

- 1,800+ flight hours in [aircraft type]; instructor pilot on [type].
- Led aviation operations sections of up to 40 personnel.
- Planned and executed mission-essential aviation operations across [theaters]; clean safety record.

Education

M.S. Data Science – University of Texas at Austin · 2025 – 2027 (expected)

Coursework: predictive modeling, statistical learning, optimization, deep learning. GPA [optional].

B.S. Computer Science – [University] · [Years]

[Honors / GPA if strong]

Skills

****Languages:**** Python (expert), Go, TypeScript, SQL, Bash, some Rust.

```

**Data:** PostgreSQL, ClickHouse, Polars, scikit-learn, PyTorch, Apache Airflow.

**Infrastructure:** Docker, Kubernetes, Terraform, GitHub Actions, AWS
(ECS, RDS, S3, Lambda), Caddy.

**Practices:** TDD, code review, on-call rotations, technical writing,
mentoring.

## Projects

### apm-class-bot

*Discord bot for tracking peer-graded deadlines in a UT MSDS course.
Python · discord.py · fly.io. [github.com/yourhandle/apm-class-bot]*

### howtos

*A personal library of opinionated technical how-tos covering VPS
provisioning, containerized deploys, scientific Python, and more.
Markdown + automated PDF rendering. [github.com/yourhandle/howtos]*

## Certifications

- CompTIA Security+ (CE) · valid through [date]
- [Other certifications]

## Honors

- [Any awards, distinctions, scholarships worth listing]

```

Style rules that come from the resume-writing canon, not LaTeX:

- **Lead each bullet with a strong verb in the past tense** ("Led", "Designed", "Reduced", "Shipped"), not "Responsible for" or "Worked on."
- **Quantify everything you can.** "p99 latency from 1,400ms to 220ms" beats "improved latency." "Cut monthly AWS spend 38%" beats "reduced costs."
- **Three to five bullets per role.** Past-five gets diminishing returns; one or two looks thin.
- **Reverse chronological** within each section.
- **Aim for one page** for industry. Two is acceptable if you have >10 years experience. Three is a CV pretending to be a resume — cut.

7. The build script

The build script does five things:

1. Merge the base `resume.md` with the chosen variant overlay (if any).
2. Inject the current git SHA and date into the YAML metadata.
3. Run pandoc with the right template and `--pdf-engine=xelatex`.
4. Produce PDF, docx, and HTML in `dist/`.
5. Print a summary.

Write it in Python (uv-managed). Bash is fine for the simplest case, but the overlay merge benefits from real data structures.

`scripts/build.py`

```

#!/usr/bin/env python3
"""Build resume variants from markdown sources.

Usage:
    uv run scripts/build.py base
    uv run scripts/build.py sde-roles
    uv run scripts/build.py ats-plain --format docx
    uv run scripts/build.py --all
"""

from __future__ import annotations

import argparse
import subprocess
import sys
from datetime import date
from pathlib import Path

import yaml

ROOT = Path(__file__).resolve().parent.parent
DIST = ROOT / "dist"
TEMPLATES = ROOT / "templates"
VARIANTS = ROOT / "variants"
BASE = ROOT / "resume.md"

def git_sha() -> str:
    try:
        out = subprocess.check_output(
            ["git", "rev-parse", "--short", "HEAD"], cwd=ROOT, text=True
        )
        return out.strip()
    except subprocess.CalledProcessError:
        return "uncommitted"

def split_front_matter(text: str) -> tuple[dict, str]:
    """Return (yaml_dict, body_markdown)."""
    if not text.startswith("---\n"):
        return {}, text
    _, fm, body = text.split("---\n", 2)
    return yaml.safe_load(fm) or {}, body

def merge_overlay(base_text: str, overlay_text: str) -> str:
    """Overlay file replaces sections of the base by H2 heading match.

    The overlay file's YAML metadata is merged on top of the base's
    (overlay wins). The overlay's H2 sections replace the base's H2
    sections of the same name. Sections in the base that aren't in the
    overlay survive unchanged.
    """
    base_meta, base_body = split_front_matter(base_text)
    overlay_meta, overlay_body = split_front_matter(overlay_text)

    merged_meta = {**base_meta, **overlay_meta}

    base_sections = split_sections(base_body)

```

```

overlay_sections = split_sections(overlay_body)

for name, content in overlay_sections.items():
    base_sections[name] = content

body = "".join(f"## {name}\n{content}" for name, content in base_sections.items())

return "---\n" + yaml.safe_dump(merged_meta, sort_keys=False) + "---\n" + body

def split_sections(body: str) -> dict[str, str]:
    """Split a markdown body into {section_name: section_content} by H2."""
    sections: dict[str, str] = {}
    current_name: str | None = None
    current_lines: list[str] = []
    for line in body.splitlines(keepends=True):
        if line.startswith("## "):
            if current_name is not None:
                sections[current_name] = "".join(current_lines)
                current_name = line[3:].strip()
                current_lines = []
            else:
                current_lines.append(line)
    if current_name is not None:
        sections[current_name] = "".join(current_lines)
    return sections

def render(variant: str, fmt: str) -> Path:
    """Render the given variant to the given format. Returns output path."""
    DIST.mkdir(exist_ok=True)

    base_text = BASE.read_text()
    if variant == "base":
        merged = base_text
    else:
        overlay_path = VARIANTS / f"{variant}.md"
        if not overlay_path.exists():
            sys.exit(f"variant not found: {overlay_path}")
        merged = merge_overlay(base_text, overlay_path.read_text())

    # Inject build metadata
    meta, body = split_front_matter(merged)
    meta["date"] = date.today().isoformat()
    meta["git-sha"] = git_sha()
    merged = "---\n" + yaml.safe_dump(meta, sort_keys=False) + "---\n" + body

    # Stage merged source for pandoc
    staged = DIST / f"_{variant}.md"
    staged.write_text(merged)

    # Pick template + engine per format
    out = DIST / f"resume-{variant}.{fmt}"
    is_ats = "ats" in variant
    template = TEMPLATES / ("ats.tex" if is_ats else "altacv.tex")

    cmd = ["pandoc", str(staged), "-o", str(out)]
    if fmt == "pdf":
        cmd += [

```

```

        "--pdf-engine=xelatex",
        f"--template={template}",
    ]
    elif fmt == "docx":
        # docx ignores LaTeX template; uses a reference docx for styling
        ref = TEMPLATES / "reference.docx"
        if ref.exists():
            cmd += [f"--reference-doc={ref}"]
    elif fmt == "html":
        cmd += ["--standalone", "--css=templates/resume.css"]
    elif fmt == "txt":
        cmd += ["--to=plain", "--wrap=none"]
    else:
        sys.exit(f"unknown format: {fmt}")

    subprocess.run(cmd, check=True)
    return out

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("variant", nargs="?", default="base")
    parser.add_argument("--format", "-f", default="pdf",
                        choices=["pdf", "docx", "html", "txt"])
    parser.add_argument("--all", action="store_true",
                        help="Render all variants in all formats")
    args = parser.parse_args()

    if args.all:
        variants = ["base"] + [p.stem for p in VARIANTS.glob("*.md")]
        formats = ["pdf", "docx", "html", "txt"]
        for v in variants:
            for f in formats:
                out = render(v, f)
                print(f"  built {out.relative_to(ROOT)}")
        return

    out = render(args.variant, args.format)
    print(f"built {out.relative_to(ROOT)}")

if __name__ == "__main__":
    main()

```

Makefile

Make is older than dirt but it gives you tab-completion and a memorable interface. Targets are short on purpose; they're what you type from muscle memory.

```
.PHONY: all pdf docx html txt sde ds pm ats clean

PYTHON := uv run --with pyyaml python

all:
    $(PYTHON) scripts/build.py --all

pdf:
    $(PYTHON) scripts/build.py base --format pdf

docx:
    $(PYTHON) scripts/build.py base --format docx

html:
    $(PYTHON) scripts/build.py base --format html

txt:
    $(PYTHON) scripts/build.py base --format txt

sde:
    $(PYTHON) scripts/build.py sde-roles --format pdf
    $(PYTHON) scripts/build.py sde-roles --format docx

ds:
    $(PYTHON) scripts/build.py data-science --format pdf
    $(PYTHON) scripts/build.py data-science --format docx

pm:
    $(PYTHON) scripts/build.py product-manager --format pdf

ats:
    $(PYTHON) scripts/build.py ats-plain --format pdf
    $(PYTHON) scripts/build.py ats-plain --format txt

clean:
    rm -rf dist/
```

Build with `make sde` or `make all`. Tab-complete in bash with `make <Tab>`.

TIP. If you prefer `just` over `make` (and you should consider it — `just` is a clean reimplementation with sane defaults), the conversion is trivial. The targets become recipes; the same commands run. Use whichever you actually have installed.

8. Multiple variants for different applications

The reason this whole exercise pays off is **variants**. A real job search means tailoring the resume per role — moving relevant experience up, rewriting the summary, swapping a couple of bullets to emphasize the right tech. Without an overlay system, you copy-paste between files and they drift. With overlays, the **base is canonical** and variants are small focused diffs.

The overlay pattern

An overlay is a markdown file with the same front-matter + sections structure as the base. The build script's `merge_overlay` function (above) does two things:

1. **Merges YAML.** Overlay keys overwrite base keys.
2. **Replaces H2 sections by name.** If the overlay has a `## Summary`, it replaces the base's `## Summary` entirely. Sections in the base that the overlay doesn't mention are kept as-is.

That's it. No template syntax, no `{% if %}` logic, no Jinja2. The overlay file is itself a valid markdown resume — you can preview it standalone if you want.

Example: `variants/sde-roles.md`

This overlay is for SWE/SDE-focused applications. It rewrites the summary, reorders skills to lead with backend languages, and adds an extra Open Source bullet to projects.

```
---
tagline: "Senior Software Engineer · Backend & Distributed Systems"
---

## Summary

Backend engineer with 8 years building distributed systems in Python and
Go. Recent focus on data infrastructure, streaming pipelines, and
zero-downtime migrations. Active-duty Army aviator transitioning to
full-time engineering. Active MSDS at UT Austin (graduating 2027).

## Skills

**Languages:** Python (expert), Go (proficient), Rust (learning), TypeScript, SQL, Bash.

**Backend:** PostgreSQL, ClickHouse, Redis, gRPC, REST, GraphQL, message
queues (NATS, Kafka).

**Infrastructure:** Docker, Kubernetes, Terraform, GitHub Actions, AWS
(ECS, RDS, S3, Lambda), Caddy.

**Practices:** Test-driven development, code review, on-call rotations,
incident postmortems, technical writing.

## Projects

### apm-class-bot

*Discord bot tracking peer-graded deadlines for an MSDS course. Python ·
discord.py · fly.io. Production traffic from 90+ students.
[github.com/yourhandle/apm-class-bot]*

### howtos

*Personal library of opinionated technical how-tos: VPS provisioning,
containerized deploys, scientific Python. Markdown + WeasyPrint
build pipeline. [github.com/yourhandle/howtos]*

### Open Source Contributions

*Patches accepted in [list]. Maintainer of [project name].*
```

Sections not mentioned (Experience, Education, Certifications) come from the base unchanged.

Example: `variants/data-science.md`

```
---
tagline: "Data Scientist · ML & Statistical Modeling"
---

## Summary

Data scientist and engineer blending production software experience with
applied ML. MSDS at UT Austin focusing on predictive modeling and deep
learning. Background in distributed systems gives a production-shaped
view of ML – pipelines that actually ship and stay shipped, not just
notebooks.

## Skills

**Modeling:** scikit-learn, PyTorch, statsmodels, XGBoost, Bayesian
methods (PyMC), causal inference.

**Data engineering:** Polars, pandas, DuckDB, Apache Airflow, dbt,
PostgreSQL, ClickHouse.

**Languages:** Python (expert), SQL (expert), R (working), Go, Bash.

**Infrastructure:** Docker, Kubernetes, GitHub Actions, AWS (SageMaker,
ECS, RDS, S3), MLflow, Weights & Biases.

**Practices:** Reproducible research, model evaluation, A/B testing,
production ML deployment.
```

Branch-per-application as an alternative

The overlay approach is the **default**. But you can also use **git branches per application** for one-off heavy customization where an overlay would be unwieldy:

```
git switch -c for/anthropic-staff-eng
# heavy edits – rewrite multiple sections, reorder experience
make pdf
# commit, build, ship
```

When to use branch vs overlay:

Customization	Use
Swap summary + reorder skills	Overlay
Add 1-2 sentences to a bullet	Overlay
Rewrite three sections, reorder roles, add a new section	Branch
Strip experience down for a senior IC pivot	Branch

For most applications, an overlay is enough. Branches are for the once-a-quarter "this is a stretch role at a top company, I want the whole resume rephrased for them" case.

TIP. Name application branches `for/<company>-<role>` consistently — `for/anthropic-staff-eng`, `for/stripe-senior-data-eng`. Don't delete them after applying; the branch is the record of what you submitted. If you get an interview, you'll want to re-read the exact version you sent. `git switch for/anthropic-staff-eng && make pdf` rebuilds it identically.

9. ATS considerations

An Applicant Tracking System (ATS) is the software a company runs your resume through before a human ever sees it. It does two things:

1. **Parses your resume into structured fields** (name, contact, work history, education, skills).
2. **Scores it against a job description** by keyword match and other heuristics.

If the parser fails — if it can't figure out where your jobs end and your education begins — your score plummets and a recruiter never sees the application. **The ATS is the gatekeeper.** It doesn't matter how beautiful your AltaCV PDF is if Workday/Greenhouse/Lever extracted it as one long blob of unstructured text.

Rules for an ATS-survivable variant

1. **Single column.** Multi-column PDFs are the #1 cause of ATS parse failures. The parser reads in column-major order and your skills section gets interleaved with your job titles.
2. **Standard section headings.** Use exactly `Experience` (or `Work Experience`), `Education`, `Skills`, `Projects`, `Certifications`. Avoid clever headings like "Where I've Done The Work" — the ATS hashes against expected names.
3. **No graphics, icons, or photos.** Anything non-text is dropped (best case) or corrupts adjacent text (worst case).
4. **No tables for layout.** Tables for genuinely tabular data (like a publications list) are fine; tables to create a two-column look are a parse killer.
5. **Plain Unicode characters only.** No fancy bullets (◦ is fine; ☒ is risky). No em-dashes from font glyphs that don't have a Unicode codepoint. No ligatures.
6. **Standard fonts.** The PDF should embed Latin/Helvetica/Times or similar. Avoid icon fonts and obscure custom faces. If a glyph isn't in the parser's font knowledge, it gets dropped.
7. **Real text, not "text shaped from outlines."** Some templates render text as vector outlines for visual perfection. The ATS extracts nothing. Test by selecting the text in the PDF — if you can't copy-paste it as text, the ATS can't either.
8. **Plain bullet lists.** Markdown's `-` becomes the right LaTeX command via pandoc. Don't introduce custom list environments.

Maintain two variants

This is the load-bearing recommendation:

Variant	Template	Use case
Human variant (default)	AltaCV (two-column, accent color, icons)	Direct email to a hiring manager. Portfolio page. Sharing with a recruiter you have a relationship with.
ATS variant	Minimal single-column template (<code>templates/ats.tex</code>)	Every upload portal: Workday, Greenhouse, Lever, iCIMS, Taleo. Anything where you click "Upload Resume" on a corporate careers page.

Default to uploading the ATS variant. Email the human variant only when you know it'll reach a human directly.

The ATS-plain overlay

`variants/ats-plain.md` is minimal — most of the override happens at the **template** level (single-column LaTeX), not the content level. The overlay mostly exists to flip the template:

```
---
template: ats
tagline: ""
---
```

The build script (§7) picks the template based on whether the variant name contains "ats". For more sophistication, add a `template:` key to the YAML and have the script read it.

A minimal ATS-survivable LaTeX template

`templates/ats.tex` — single column, no icons, no color, no nonsense. Pandoc fills in the variables.

```

\documentclass[11pt,letterpaper]{article}
\usepackage[margin=0.7in]{geometry}
\usepackage{enumitem}
\usepackage{titlesec}
\usepackage{hyperref}
\usepackage{fontspec}
\setmainfont{TeX Gyre Termes} % Times-equivalent; ATS-friendly

\setlist{nosep,leftmargin=*}
\titleformat{\section}{\large\bfseries}{}{0pt}{}[\titlerule]
\titlespacing*{\section}{0pt}{8pt}{4pt}

\pagestyle{plain}
\setlength{\parindent}{0pt}

\begin{document}

\begin{center}
{\Large\bfseries $name$} \[\[2pt]
$email$ \if(phone)$\textbar\ $phone$ $endif$\if(location)$\textbar\ $location$ $endif$\if(homepage)$\homepage$\ $endif$\if(github)$\textbar\ $github$\ $endif$\if(linkedin)$\textbar\ $linkedin$\endif$
\end{center}

\vspace{4pt}
$body$

\vfill
\begin{center}
{\footnotesize Version $git-sha$ \textbullet\ Built $date$}
\end{center}

\end{document}

```

That's the whole template. It produces a single-column, plain-text-equivalent PDF that every ATS in the industry can parse correctly.

Test your ATS variant

Before submitting to any portal, **dry-run the parse**:

1. Open the rendered PDF.
2. **Select all (Ctrl+A) and copy.**
3. Paste into a text editor.

If the result reads like a structured resume top-to-bottom, you're fine. If sections are interleaved, characters are missing, or whitespace has collapsed, the ATS will see the same garbage.

Also useful: **resume-checker** tools that simulate ATS parsing. Most are commercial and predatory but a few are free. Run your resume through one once a year. Don't pay them; the free tier is enough.

IMPORTANT. SOME ATS PORTALS ALSO LET YOU PASTE PLAIN TEXT DIRECTLY. When given the choice, **PASTE THE PLAIN TEXT** (output of `pandoc --to=plain --wrap=none`) — the parse step is skipped entirely. The job description matching still runs against the same text, so your keyword density still matters.

10. docx output for HR portals

A non-trivial number of HR portals reject PDFs and demand `.docx`. This is dumb but real. The pandoc fix is one flag.

```
pandoc resume.md -o dist/resume.docx
```

That works without a reference document and produces a passable Word file. To get something that looks like your resume rather than pandoc's defaults, supply a **reference document** — a Word file whose styles pandoc will use:

```
pandoc resume.md --reference-doc=templates/reference.docx -o dist/resume.docx
```

To create `templates/reference.docx`:

1. Run `pandoc -o reference.docx --print-default-data-file reference.docx` to get pandoc's default.
2. Open it in Word / LibreOffice.
3. Edit the styles (Heading 1, Heading 2, Normal, etc.) to match how you want your resume to look.
4. Save it back.

Now `pandoc --reference-doc=...` applies those styles to your resume content.

CAUTION. DON'T MANUALLY EDIT THE RESUME.DOCX FILE. It's a build artifact. If you edit the docx directly, your changes are lost the next time you build, and you lose the source-of-truth property that motivated this whole setup. If the docx is wrong, the fix is in `resume.md` or `templates/reference.docx`, not in the output.

11. Dev Container for resume building

TeX Live is ~5 GB. WeasyPrint pulls native dependencies. Pandoc is fine but the LaTeX engine wants matched fonts. Installing all of this on your host is doable but it's also exactly what Dev Containers are for: reproducible builds, isolated from the host, identical on every machine you own.

`.devcontainer/devcontainer.json`

```
{
  "name": "Resume Builder",
  "build": { "dockerfile": "Dockerfile" },
  "customizations": {
    "vscode": {
      "extensions": [
        "yzhang.markdown-all-in-one",
        "DavidAnson.vscode-markdownlint",
        "James-Yu.latex-workshop",
        "tamasfe.even-better-toml",
        "redhat.vscode-yaml"
      ],
      "settings": {
        "editor.wordWrap": "on",
        "files.eol": "\n"
      }
    }
  },
  "postCreateCommand": "uv sync || true",
  "remoteUser": "vscode"
}
```

`.devcontainer/Dockerfile`

```
FROM mcr.microsoft.com/devcontainers/python:3.12-bookworm

# Pandoc – pull the latest stable from upstream rather than apt's older version
ARG PANDOC_VERSION=3.1.13
RUN curl -fsSL "https://github.com/jgm/pandoc/releases/download/${PANDOC_VERSION}/pandoc-${PANDOC_VERSION}-1-amd64.deb" -o /tmp/pandoc.deb \
    && dpkg -i /tmp/pandoc.deb \
    && rm /tmp/pandoc.deb

# TeX Live – full install is large; this set is what AltaCV + most pandoc templates need
RUN apt-get update && apt-get install -y --no-install-recommends \
    texlive-xetex \
    texlive-fonts-recommended \
    texlive-fonts-extra \
    texlive-latex-extra \
    texlive-luatex \
    fonts-roboto \
    fonts-source-code-pro \
    lmodern \
    && rm -rf /var/lib/apt/lists/*

# WeasyPrint native deps (for the optional HTML/CSS path)
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpango-1.0-0 \
    libpangoft2-1.0-0 \
    libcairo2 \
    libgdk-pixbuf-2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# uv – fast Python tooling, installed for the vscode user
USER vscode
RUN curl -LsSf https://astral.sh/uv/install.sh | sh
ENV PATH="/home/vscode/.local/bin:${PATH}"

# AltaCV – install to user TEXMF
RUN mkdir -p /home/vscode/texmf/tex/latex/altacv \
    && curl -fsSL "https://raw.githubusercontent.com/liantze/AltaCV/main/altacv.cls" \
    -o /home/vscode/texmf/tex/latex/altacv/altacv.cls

WORKDIR /workspaces
```

Using the container

In VS Code: open the resume folder, accept the "Reopen in Container" prompt. First build takes ~5 minutes (pandoc download + TeX install). Subsequent opens are instant. From then on:

```
make pdf      # inside the container terminal
make sde
make ats
```

The output `dist/` appears on your host filesystem because the workspace is bind-mounted. Hand the PDF off to a recruiter from your host file manager.

TIP. Build the container image once, push it to your personal container registry (GHCR), and reference the image directly in `devcontainer.json` for subsequent machines:

```
json { "image": "ghcr.io/yourhandle/resume-builder:latest" }
```

Now opening the project on a new laptop is instant — no 5-minute rebuild. The image rebuild is something you do **ONCE A YEAR** when bumping pandoc / TeX Live versions.

12. Git workflow for a resume

The `git-for-solo-devs` workflow applies straightforwardly here, with a few resume-specific conventions:

Branch model

- `main` is the **general / default variant** of your resume. The state you'd ship if you had ten minutes and no specific target role.
- `for/<company>-<role>` branches are per-application. Heavy customization lives here. Never merged back.
- **Overlays in `variants/`** are committed to `main`. They're long-lived first-class artifacts, not per-application throwaways.

Commit messages

Treat the resume like a real codebase. Use conventional-ish messages — they double as a personal changelog.

```
add senior data engineer role at acme
quantify p99 latency improvement in beta inc bullet
rewrite summary for sde-roles overlay to emphasize backend
bump education end date (msds graduation projected 2027)
fix typo in altacv template (s/heigth/height/)
```

When you're applying:

```
for/anthropic-staff-eng: rewrite first bullet of acme experience
for/anthropic-staff-eng: shorten military experience to 2 bullets
for/anthropic-staff-eng: ship – submitted 2026-05-27
```

That last commit type ("ship") is a habit worth building. It marks **what version actually got sent**. When the recruiter asks about something on your resume two weeks later, `git log --all | grep ship` tells you which version they're looking at.

PR yourself

For non-trivial edits (new role, restructured sections), open a PR against your own repo, even though no one else will review it. Why:

- **Self-review on the diff catches things you missed.** Reading your resume as a diff is qualitatively different from reading the rendered PDF.
- **PR descriptions become changelogs.** "What changed since I last updated the resume?" is a question you'll ask once a year.

- **The CI workflow runs** (see below) and confirms the build still works before you merge.

A solid CI workflow for a resume repo: `pandoc → PDF`, fail on warning. Catches missing variables, broken templates, and template syntax errors before you merge.

```
# .github/workflows/build.yml
name: build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    container: ghcr.io/yourhandle/resume-builder:latest
    steps:
      - uses: actions/checkout@v4
      - run: make all
      - uses: actions/upload-artifact@v4
        with:
          name: resume-pdfs
          path: dist/
```

The downloadable artifact at the end is occasionally useful — sometimes you want the latest PDF and you're on a machine without the container. The CI artifact is one click away.

13. Privacy: don't ship PII to a public repo

Your resume contains your email, phone, address, military service details, employment history, and education record. **None of that belongs in a public repo.** Two reasonable patterns:

Pattern A: Private repo (recommended)

Keep the entire resume repo private. Simple, clean, no risk of accidentally pushing PII. GitHub gives unlimited private repos on the free tier — there's no real reason not to.

```
gh repo create resume --private
```

The whole flow above just works. The build artifact (PDF) can be uploaded to anywhere you need it; the source stays private.

Pattern B: Public repo + local-only personal data file

Some people want their resume source public (as a portfolio piece, an open-source-software-style flex, or because they want it discoverable). The compromise:

- The repo is **public**.
- Real PII lives in a **gitignored** `personal.yaml` file on your machine only.
- The committed `resume.md` uses placeholders like `[Your Name]`, `[email]`, etc.
- The build script merges `personal.yaml` into the metadata at build time.

```
# in scripts/build.py, before pandoc invocation
personal_path = ROOT / "personal.yaml"
if personal_path.exists():
    personal = yaml.safe_load(personal_path.read_text())
    meta = {**meta, **personal}
```

`personal.yaml` is in `.gitignore`:

```
personal.yaml
```

The downside: **anyone cloning your repo will get a resume with placeholders**. That's the entire point. Only your local machine has the real data.

CAUTION. DO NOT COMMIT `PERSONAL.YAML` "JUST ONCE TO TEST." That's how every secret-leak post-mortem starts. The file is in `.gitignore` from the first commit; verify with `git status` before every push that `personal.yaml` is untracked. If you ever accidentally commit it, treat it as a credential leak: rotate what's rotatable (phone is hard, address is hard, email is doable), and consider rewriting history to scrub the file (`git filter-repo` — but the data may already be cached by mirrors).

Use Pattern A. Private repos cost nothing and avoid the whole class of accidents. Pattern B exists for the (rare) case where you really want the source public.

14. Versioning and dating the output

Every PDF you ship should carry a **version identifier and build date** in the footer. Reasons:

- A recruiter asks about "the resume you sent us" three weeks later. The footer tells you which version.
- You realize a bug after applying ("I left in the placeholder for the company name"). The footer SHA tells you exactly which commit shipped.
- You're presenting a portfolio of past applications later in your career — the footer date is the record.

Inject git SHA and date into pandoc metadata

The build script (§7) already does this — it sets `meta["date"]` and `meta["git-sha"]` before invoking pandoc. The template references them:

```
% in templates/altacv.tex, in the footer area
\renewcommand{\footrulewidth}{0pt}
\fancyfoot[L]{\footnotesize Version $git-sha$}
\fancyfoot[R]{\footnotesize Built $date$}
```

And in the ATS template (`templates/ats.tex`, shown above), at the bottom of the document:

```
\vfill
\begin{center}
{\footnotesize Version $git-sha$ \textbullet Built $date$}
\end{center}
```

The footer looks like this in the rendered PDF:

```
Version 4f3a9c2 · Built 2026-05-27
```

Six-character SHA + ISO date. Small font, gray, bottom margin. Nobody notices it but you, and you only need it when you need it.

When you ship: tag the commit

After building and submitting, **tag the commit** so it's easy to recover:

```
git tag -a "for/anthropic-staff-eng" -m "submitted 2026-05-27"  
git push --tags
```

Now `git checkout for/anthropic-staff-eng` rebuilds the exact PDF you submitted. The SHA in the footer matches the tag. Two years later, if someone asks "you mentioned project X on your resume when you applied — could you walk through it?", you have an exact rebuild.

15. Anti-patterns

Things to actively not do, paired with what to do instead.

Resume lives in Microsoft Word / Google Docs

The problem: Binary diffs are useless. You can't see what changed. Multiple variants mean copy-paste, which means drift. Formatting fights you. Auto-correction silently changes content. The doc lives on someone else's server.

The fix: Plain markdown, version-controlled. This whole how-to.

Multiple resume files in a folder (`resume-anthropic.md`, `resume-stripe.md`, ...)

The problem: Copy-paste between files. When you fix a typo, you fix it in one and forget the others. When you add a new role, you have to manually port the bullet to every variant.

The fix: One base + overlays. The base is canonical. Overlays are diffs. The build script merges them. No file ever drifts from another.

No source control

The problem: You can't answer "what did my resume look like when I applied to X in 2024?" You lose accidental edits. You have no record of what was sent.

The fix: Git. Even if it's just `git init && git commit -am "first" && git add -A && git commit -am "today"` once a week, you've recovered all the history. Better: tag every submission.

Screenshotting parts of the resume into other documents

The problem: You paste a chunk of your resume into a cover letter / portfolio site / LinkedIn About section. Now there are N sources of truth. The original updates; the screenshots don't.

The fix: Treat the resume as the canonical source and have other documents reference it (or render from it). If your portfolio site needs your work history, generate the work history HTML from `resume.md` at build time — same pandoc, different template.

Letting Canva / Resume.io / Zety own your data

The problem: Lock-in. Their export format changes. They paywall the template you used. They go out of business. Your source isn't owned by you.

The fix: Markdown source. You own the file. Renderers come and go; the source survives.

Optimizing for the upload portal's preview, not the actual ATS parse

The problem: The upload portal often shows a visual preview that looks fine. You assume the parse worked. It didn't. Your application is scored low and you never know why.

The fix: Always **dry-run the parse** by copy-pasting from the PDF (\$9). If the text comes out garbled, the ATS sees garbage too. If the portal accepts plain text directly, paste the plain-text version.

A two-column "creative" resume for an SDE role

The problem: ATS parse failure rate on two-column PDFs is meaningfully high. You'll never know if you got filtered out — you'll just stop getting callbacks.

The fix: Maintain both variants. Use the single-column ATS variant for upload portals. Use the AltaCV variant for direct human handoff.

Editing the rendered PDF / docx directly

The problem: You break the source-of-truth chain. Edits are lost on next build. You forget where the "real" copy is.

The fix: All edits in `resume.md` or `variants/*.md`. The rendered files are artifacts; never edit them.

Letting the resume go stale because "I'm not actively job hunting"

The problem: When you suddenly need the resume — referral, recruiter ping, surprise opportunity — it's nine months out of date and you spend a weekend rewriting it under time pressure.

The fix: Quarterly update ritual. Open a calendar event. Spend 30 minutes once a quarter adding what happened, refining bullets, deleting irrelevant detail. The cumulative cost is two hours per year; the recovery cost from a stale resume is several days.

16. Alternatives considered

The toolchains and patterns you might use instead, and when each one would be the right call.

Canva / Resume.io / Zety / Novoresume

Verdict: Use only if you need a resume tomorrow and have never written one.

Why not the default: Lock-in. Subscription costs. You can't diff. Their templates trap you in two-column layouts that ATS systems mangle. Your "source" is in their database, not on your disk.

When to reconsider: Genuinely one-off use — graduating senior with no resume yet, applying to a deadline tomorrow. Build one in Canva, ship it, then port it to markdown the following week. Don't let the one-off become permanent.

LinkedIn-as-resume

Verdict: Useful as a companion artifact, not as a primary.

Why not the default: LinkedIn owns the layout, the visual identity, and what fields exist. You can't control what's emphasized. You can't tailor per role. Many recruiters and ATS systems still want a real PDF — LinkedIn's "Download as PDF" produces a rendered version that's intentionally bare to push you back to the platform.

When to use it: Always keep it updated. It's how recruiters find you. But the PDF you upload to portals is the markdown-rendered one, not the LinkedIn PDF.

Writing LaTeX directly (no pandoc)

Verdict: Use if you already know LaTeX deeply and write academic CVs frequently.

Why not the default: The source is LaTeX, which is far less ergonomic than markdown for content editing. Inserting a new bullet requires remembering `\cvevent`, `\divider`, etc. Pandoc keeps the source human-readable.

When to reconsider: Academic CVs with very specialized formatting (publications lists with custom rendering, complex tables, math). At that point, writing LaTeX directly may be faster than fighting pandoc's template system.

HTML/CSS direct (WeasyPrint without pandoc)

Verdict: Reasonable second-best.

Why not the default: Typography is good but not LaTeX-grade. Tracking, kerning, hyphenation, and microtypography are all weaker. For a 1-page resume read in 20 seconds it's fine; for a polished portfolio piece it's noticeably less crisp.

When to reconsider: You're already using WeasyPrint in this repo (see `scripts/build-pdfs.py`) and want one fewer toolchain. You're comfortable in CSS and want full layout control. The TeX install is a hard blocker.

JSON Resume + themed renderer

Verdict: Excellent for structured-data workflows; fine as a default if you don't mind Node.

Why not the default: Theme ecosystem quality is uneven. Modifying a theme means reading someone else's Handlebars/Pug. Most people don't want that.

When to use it: You'll programmatically generate the resume from other data (a portfolio CMS, an HR system, a personalization pipeline). The strict schema is a real benefit there. Also genuinely useful if you want to A/B test 5+ visual styles cheaply.

Typst

Verdict: Watch this space — it's the most promising new entrant.

Why not the default yet: Typst is a modern typesetting system (think "LaTeX but designed in this decade") with much cleaner syntax and faster compilation. As of 2026-05 it's mature enough for real use, and a few good resume templates exist. The reasons it's not the default here: the ecosystem is still smaller than LaTeX's, ATS-aware templates are less battle-tested, and pandoc support is limited (it's a separate toolchain, not pandoc-rendered).

When to reconsider: Late-2026 / 2027. If Typst templates for resumes mature and pandoc gains first-class Typst output, it could displace LaTeX here. Re-evaluate this section at the annual sweep.

`cvitae` / `awesome-cv` / **other LaTeX template families**

Verdict: Drop-in alternatives to AltaCV.

Why not the default: AltaCV is the best-looking modern template I've seen and is actively maintained. The others (`awesome-cv`, `cvitae`, `friggeri-cv`) range from "fine" to "abandoned." Use AltaCV unless its specific look doesn't suit you.

Comparison table

Tool	Source format	Typography	ATS-friendly	Install footprint	Maintenance
pandoc + LaTeX (AltaCV)	Markdown + YAML	Excellent	Needs separate variant	Large	Low (once set up)
pandoc + LaTeX (moderncv)	Markdown + YAML	Excellent	Reasonable	Medium	Low
WeasyPrint	Markdown + CSS	Good	Reasonable	Small	Medium (CSS upkeep)
JSON Resume	YAML/JSON	Theme-dependent	Theme-dependent	Small	Medium (theme drift)
Typst	Typst syntax	Excellent	Template-dependent	Small	Watch — improving fast
Direct LaTeX	LaTeX	Excellent	Template-dependent	Large	High (LaTeX-heavy edits)
Canva / Resume.io	Their proprietary	Good (but locked)	Often poor	None	N/A — service goes down, you're stuck
Word / Google Docs	.docx / .gdoc	Mediocre	Mediocre	None	High — drift and manual sync

17. Cheat sheet

Print this section. Tape it next to your monitor.

One-time setup

```
# Create repo
gh repo create resume --private --clone
cd resume

# Scaffold structure
mkdir -p variants templates scripts .devcontainer dist
touch resume.md variants/sde-roles.md variants/ats-plain.md
touch templates/altacv.tex templates/ats.tex
touch scripts/build.py Makefile .gitignore

# Install AltaCV (host install only; the Dev Container does this for you)
mkdir -p ~/texmf/tex/latex/altacv
curl -fsSL "https://raw.githubusercontent.com/liantze/AltaCV/main/altacv.cls" \
  -o ~/texmf/tex/latex/altacv/altacv.cls
texhash ~/texmf
```

Daily build commands

```
make pdf          # build base resume → dist/resume-base.pdf
make sde          # build SDE variant (pdf + docx)
make ds          # build data-science variant
make ats          # build ATS-safe variant (pdf + txt)
make all          # build every variant in every format
make clean        # nuke dist/

# Manual pandoc (if not using the script)
pandoc resume.md \
  --template=templates/altacv.tex \
  --pdf-engine=xelatex \
  --metadata=git-sha:"$(git rev-parse --short HEAD)" \
  --metadata=date:"$(date +%Y-%m-%d)" \
  -o dist/resume.pdf
```

Per-application workflow

```
git switch -c for/anthropic-staff-eng
# edit variants/sde-roles.md (or write a new overlay)
make sde
# review PDF visually, then:
git add -A && git commit -m "for/anthropic-staff-eng: tailor summary and skills"
make ats          # also produce the ATS variant
# submit ATS PDF to portal; email human PDF to direct contact
git commit --allow-empty -m "for/anthropic-staff-eng: ship – submitted $(date +%F)"
git tag for/anthropic-staff-eng
git push --tags
git switch main
```


YAML metadata block (top of `resume.md`)

```
---
name: "Your Name"
tagline: "Engineer · Aviator · Data"
email: "you@example.com"
phone: "+1 555 555 0100"
location: "City, State"
homepage: "yourname.dev"
github: "github.com/yourhandle"
linkedin: "linkedin.com/in/yourhandle"
accent-color: "0E5484"
photo: false
---
```

ATS-safety checklist (run before every portal upload)

- [] Single column layout (ATS variant)
- [] Standard headings: `Experience`, `Education`, `Skills`, `Projects`, `Certifications`
- [] No graphics / icons / photos
- [] No tables for layout (only for genuinely tabular data, if any)
- [] All bullet points are plain `-` or `•`
- [] PDF text is **selectable** (Ctrl+A copy → readable)
- [] Copy-paste extraction produces top-to-bottom prose, not interleaved sections
- [] Footer carries git SHA and date
- [] No "Lorem ipsum" / `[placeholder]` strings left over
- [] File name is `firstname-lastname.pdf` (not `resume-final-v4.pdf`)
- [] Keyword check against the JD (do the top 5 JD terms appear at least once?)

Section-by-section style rules

Section	Rule
Summary	2-3 sentences. Tailored per variant. Third-person implicit.
Experience	Reverse chronological. 3-5 bullets per role. Strong past-tense verbs. Quantify everywhere possible.
Education	Reverse chronological. Include GPA only if strong (≥ 3.7).
Skills	Grouped by category. Most-relevant category first per variant.
Projects	Real, linkable, dated. URL on every project.
Certifications	Include expiry / CE date if applicable (Sec+, AWS, etc.).
Honors	One-liners. Cut anything older than 10 years unless major.

Common pandoc flags

```
--template=PATH           # use a custom template
--pdf-engine=xelatex      # use XeLaTeX (required for fontspec / AltaCV)
--reference-doc=PATH      # docx styling source
--metadata=KEY:VALUE      # inject a key into the YAML metadata
--to=FORMAT               # explicit output format (plain, html, docx, pdf, ...)
--wrap=none               # don't wrap output lines (useful for ATS plain text)
--standalone              # produce a full document, not a fragment (HTML)
--css=PATH                # inline a CSS file (HTML output)
-V key=value              # override a template variable directly (e.g. -V
geometry:margin=0.7in)
```

Debugging the LaTeX build

```
# Run pandoc with verbose output; the LaTeX log shows up
pandoc resume.md --template=templates/altacv.tex --pdf-engine=xelatex -o dist/resume.pdf --
verbose

# Get the intermediate .tex pandoc generates (useful for diagnosing template bugs)
pandoc resume.md --template=templates/altacv.tex -o dist/resume.tex

# Then compile that .tex by hand to see the raw LaTeX errors
xelatex -output-directory=dist dist/resume.tex
```

If pandoc produces LaTeX but XeLaTeX fails, the bug is in the template. If pandoc itself fails, the bug is in the YAML or markdown structure.

Quarterly maintenance ritual

Once a quarter, on a calendar event:

- [] Add anything new from the past quarter (roles, certs, projects, honors)
- [] Refine the most-recent role's bullets (you'll have learned how to phrase them better)
- [] Cut anything stale (5+ year old projects unless major)
- [] Update the MSDS / education progress
- [] Rebuild `make all`; commit the source changes
- [] Re-run the ATS-safety checklist on the ATS variant
- [] Verify the Dev Container still builds (`docker build .devcontainer/`)

Total time: ~30 minutes per quarter. Compounds into a resume that's always interview-ready.

See also

- [git-for-solo-devs](#) — the git workflow assumed throughout, especially the branch-per-application pattern.
- [dockerized-deployments](#) — the Dev Container pattern used in §11 is the same one used for application projects there.
- [scripts/build-pdfs.py](#) — proof-of-concept for the markdown → PDF pipeline already running in this repo. Same idea (WeasyPrint variant), applied to the how-tos themselves.